UNIVERSIDAD PRIVADA DEL NORTE

FACULTAD DE INGENIERÍA

LENGUAJES DE PROGRAMACIÓN

TÍTULO

Arquitectura de software para plataforma educativa de Laboratoria: paradigmas y lenguajes de programación.

DOCENTE

Luigie Niels Sanches Salcedo

ESTUDIANTE

Yackerly Marlit López Chamba

N00539675

FECHA

11/10/2026

Introducción

Este informe nace de analizar la plataforma educativa de Laboratoria, que no es solo una página de cursos, sino un sistema donde interactúan estudiantes, instructores y empresas al mismo tiempo. El problema es que, con más de 7,000 postulantes al año, si el sistema no está bien pensado, se vuelve un desorden difícil de mantener.

Por eso el objetivo del informe es proponer una arquitectura que sea ordenada, escalable y fácil de mantener, eligiendo para cada parte el paradigma y el lenguaje que mejor se adapta. Para eso se revisaron los seis componentes principales de la plataforma, desde la gestión de usuarios hasta la colaboración en tiempo real, y se compararon opciones como programación orientada a objetos, funcional y orientada a eventos, además de lenguajes como Python, Java y JavaScript.

La idea no es decir que una tecnología es mejor que otra, sino mostrar por qué cada una tiene su lugar dentro del sistema y cómo, combinándolas bien y aplicando buenas prácticas y patrones de diseño, se puede construir una plataforma que funcione hoy y siga funcionando a futuro.

I. Componentes del sistema y paradigmas.

La plataforma de Laboratoria está dividida en seis módulos. Cada uno funciona diferente, así que necesita un tipo de programación distinto. A continuación, analizaré cada parte y explicaré que paradigma me parece el más adecuado y por qué.

1.1 Gestión de usuarios:

- **Paradigma asignado:** Programación orientada a objetos (POO)

  Se encarga de manejar a los estudiantes, instructores y empresas con sus respectivos roles.

  Sustento: Elegí POO porque permite modelar el ciclo de vida y al crearlos como clases puedo aprovechar la herencia, lo que mejora la mantenibilidad, porque si cambio algo en un tipo de usuario no afecto a los otros.

1.2 Contenido educativo:

- **Paradigma asignado:** Programación orientada a objetos (POO)

  Acá se encuentran los cursos, lecciones, proyectos y materiales.

  Sustento: Permite crear clases para cada tipo de contenido, además con el polimorfismo puedo tener una interfaz donde la plataforma los maneja a todos de la misma forma.

1.3 Evaluación:

- **Paradigma asignado:** Programación Funcional (PF)

  Este módulo se encarga de revisar las respuestas, sacar notas, promedios y ver cómo les va a los estudiantes.

  Sustento: Elegí PF porque todo esto es **transformar datos:** entra una respuesta y sale una nota, sin cambiar o afectar otras cosas. También ayuda cuando hay muchos estudiantes a la vez, porque se puede procesar más rápido.

1.4 Colaboración en tiempo real:

- **Paradigma asignado:** Programación orientada a eventos

  Aquí van las clases por Zoom, los chats y el trabajo en equipo en los proyectos. Todo esto tiene que responder al instante cuando alguien hace algo, como mandar un mensaje, entrar a una clase o compartir pantalla.

  Sustento: Conviene usar programación orientada a eventos, porque el sistema va reaccionando a lo que pasa en el momento, no sigue un orden fijo. Así se puede tener comunicación en tiempo real y todo se actualiza al momento sin tener que recargar la página.

1.5 Seguimiento de progreso:

- **Paradigma asignado: Pr**ogramación Funcional (PF)

  Este módulo lleva el control de lo que avanza cada estudiante, como los módulos que ya terminó, las horas que le dedicó y los logros que consiguió, para después generar reportes.

  Sustento: Con la programación funcional se pueden hacer funciones puras que siempre den el mismo resultado y sin complicaciones (filter/reduce), lo que hace más fácil mantener el

código y que todo sea más confiable, sobre todo con tantos estudiantes que entran cada año.

1.6 Conexión con empresas:

- **Paradigma asignado:** Programación orientada a objetos (POO)

  Aquí se conectan las empresas con las egresadas y maneja las ofertas de trabajo, las postulaciones y los perfiles.

  Sustento: Con la POO se puede ordenar mejor la lógica de cada parte y reutilizar código, lo que ayuda a la escalabilidad a medida que se suman más empresas.

II. Comparación de lenguajes de programación (Python, Java y JavaScript):

Para seleccionar el lenguaje más adecuado para cada componente de la plataforma, según cuatro criterios: rendimiento, comunidad, bibliotecas disponibles y compatibilidad con tecnologías web.

2.1 Rendimiento:

Para el proyecto se necesita un balance entre velocidad de ejecución y el tipo de tarea que hace cada módulo.

Lenguaje	Características de ejecución	Análisis en el contexto del proyecto
Python	Python es interpretado y de tipado dinámico. Sin compilación previa, así que en procesos intensivos va más lento que Java o JavaScript.	Para **Evaluación y Seguimiento de Progreso** no necesitamos velocidad extrema, sino procesamiento de datos. Y si en algún momento se vuelve cuello de botella, se optimiza con NumPy, Pandas o Cython, sin cambiar de lenguaje,
Java	Se compila a bytecode y corre sobre la JVM. La compilación JIT y su manejo de memoria lo hacen muy eficiente bajo alta carga.	Es donde mejor aguanta volumen: **Gestión de Usuarios** (más de 7,000 postulantes) y **Conexión con Empresas**. Ahí necesitamos estabilidad y soportar muchos usuarios a la vez, y Java responde.
JavaScript (Node.js)	Corre sobre el motor V8, también con JIT. Su fuerte no es el cálculo matemático, sino las operaciones de E/S y los eventos asíncronos.	Para **Colaboración en Tiempo Real** es el indicado. En Zoom, chats y trabajo colaborativo, lo que importa es no bloquear el sistema mientras esperas un mensaje o una conexión. Node.js lo maneja de forma nativa.

2.2 Comunidad de desarrollo:

- **Python** hoy por hoy tiene una de las comunidades más grandes y activas que hay. Ha crecido muchísimo en los últimos años por todo el tema de la ciencia de datos e IA, así que se encuentra de todo: librerías, tutoriales, gente respondiendo en foros. Además, es un lenguaje bastante amigable y fácil de aprender, por lo que al meter gente nueva al equipo les es fácil adaptarse.
- **Java** Si se busca algo estable y que siga vigente a largo plazo, Java es una apuesta segura. Lleva ya más de 20 años en el mercado y una comunidad muy consolidada, sobre todo en el mundo empresarial. Lo bueno que hay documentación para todo, muchos desarrolladores con experiencia y el respaldo de empresas gigantes detrás.
- **JavaScript** es la base de toda la web. Funciona en frontend y en backend con Node.js, literal todo el mundo lo usa. Su ecosistema cambia rapídisimo, y a cada rato salen herramientas nuevas, lo cual es una ventaja si queremos mantener lo último en actualización.

2.3 Bibliotecas disponibles:

- **Python:** Es perfecto para la parte de análisis y procesamiento de datos. Tiene librerías clave como Pandas, NumPy, Matplotlib y SciPy, que nos sirven directo para los módulos de evaluación y seguimiento, donde hay que calcular notas, promedios y sacar estadísticas.
- **Java:** Es excelente para el control de usuarios y la conexión con empresas. Se usa mucho a nivel empresarial y viene muy completo. Tiene frameworks como Spring Boot, que ayuda a armar un backend que aguante bastante carga, e Hibernate para manejar la base de datos de forma más fácil.
- **JavaScript:** Tiene las herramientas más populares del momento. Para el diseño de pantallas (frontend) están React, Vue o Angular, y para el servidor (backend) podemos usar Express o Next.js. Además, si el proyecto necesita funciones en tiempo real, cuenta con Socket.io para que la comunicación sea inmediata.

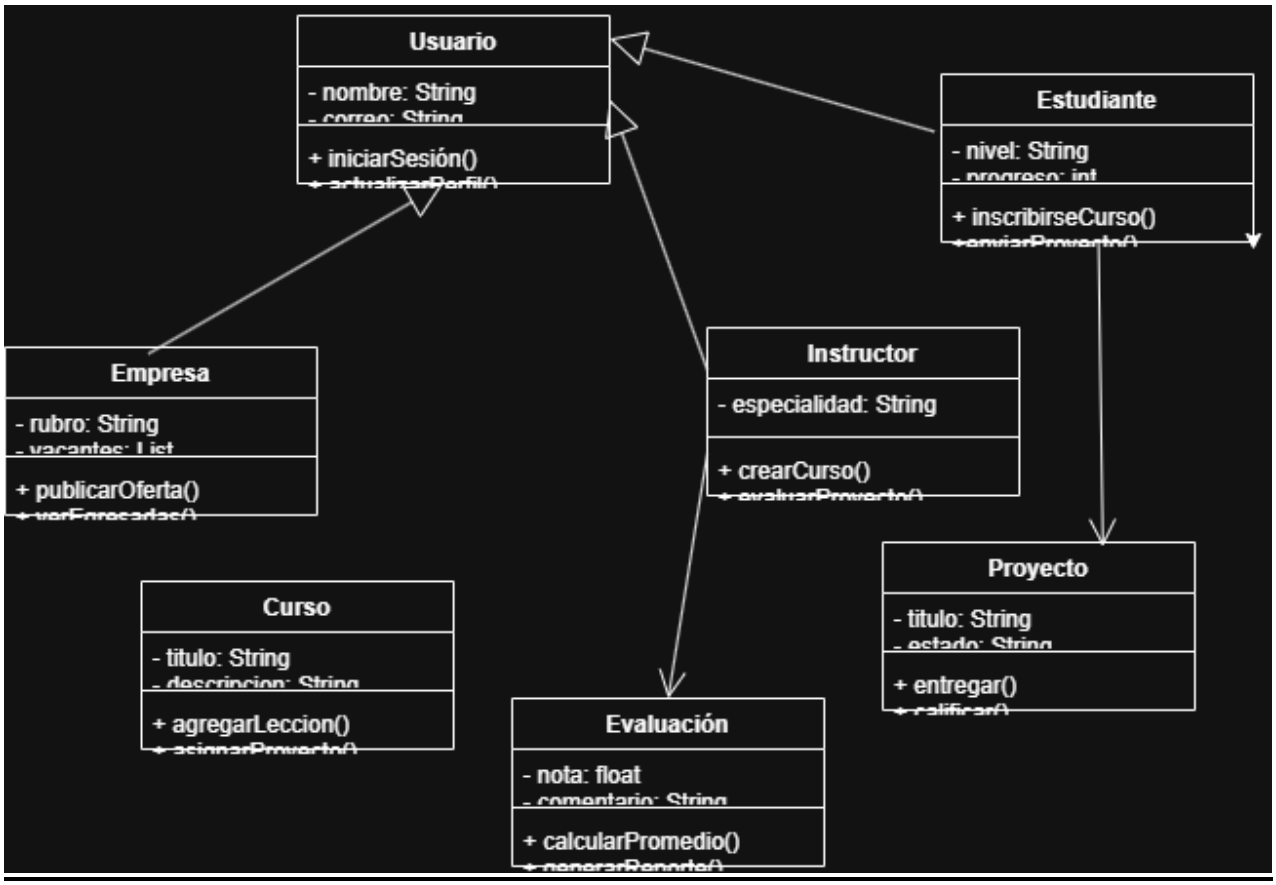
2.4 Compatibilidad con tecnologías web:

- **Python:** No corre en el navegador de forma nativa, pero se usa mucho en el backend con frameworks como Django y Flask. Estos nos permiten armar APIs REST y manejar toda la lógica del servidor sin complicaciones.
- **Java:** Sirve para crear APIs seguras y aguantar aplicaciones web que manejan alto volumen de tráfico. Tampoco se ejecuta en el navegador, pero es de las opciones más fuertes para el backend en empresas grandes usando Spring Boot.
- **JavaScript:** Es el único que corre de forma nativa en el navegador, así que es obligatorio para el frontend. Lo bueno es que con Node.js también se puede usar en el backend, por lo que puedes programar toda la aplicación (full-stack) con un solo lenguaje y hacer que las funciones en tiempo real sean más

fáciles de armar.

III. Programación orientada a objetos en gestión de perfiles

3.1 Diagrama UML: El siguiente diagrama muestra las clases y sus relaciones:



3.2 Pilares aplicados:

- **Encapsulamiento:** Los atributos están en privado (-nombre, -correo), entonces no se pueden tocar directo desde afuera. Para cambiarlos se usan métodos públicos como visualizarPerfil() o calcularPromedio(). Así se cuidan mejor los datos del usuario y no se modifica una nota por error.
- **Herencia:** Se creó una clase base que es Usuario y ahí está lo que todos tienen en común: - nombre, - correo. De esa clase salen Estudiante, Instructor y Empresa, que usan lo mismo, pero le agregan lo suyo. Esto ayuda bastante porque si hay que cambiar algo del login, solo se cambia en Usuario y no en las tres clases.
- **Polimorfismo:** Aunque todos son Usuario en general, a la hora de ejecutarse cada uno hace lo que le toca: el estudiante se escribe a un curso, el instructor lo crea y la empresa publica una vacante.

IV. Paradigmas en la arquitectura y componentes de la plataforma:

Para que todo el sistema de Laboratoria funcione bien, y aguante la gestión de postulantes, el control de las 40 horas semanales y el trabajo en tiempo real, un solo paradigma no es suficiente. El sistema se vuelve realmente sólido cuando se combinan tres: Orientado a Objetos, Funcional y Orientado a Eventos, cada uno aportando en el componente donde mejor se desempeña.

4.1 Arquitectura General:

La arquitectura de Laboratoria está separada por capas y en cada una se usa el paradigma que mejor le va. En la parte visual, que es lo que ve el usuario en el navegador, usamos eventos para que todo lo de Zoom, chats y proyectos colaborativos responda al momento. En la lógica del negocio, lo funcional nos sirve para procesar datos como notas, promedios y el avance de cada estudiante, de forma más limpia y sin que una cosa afecte a otra. Y ya en la parte de datos, lo orientado a objetos nos ayuda a modelar bien quiénes son los actores del sistema, o sea estudiantes, instructores y empresas, y también los cursos, proyectos y evaluaciones.

Así cada paradigma hace lo que mejor sabe hacer y entre los tres logramos que el sistema sea escalable, fácil de mantener y rápido.

4.2 Paradigma por componente:

Podemos notar que cada uno de los seis componentes usa el paradigma que más le conviene.

- **Gestión de usuarios, Contenido educativo y Conexión con empresas van con orientado a objetos.** Son básicamente las entidades del sistema: estudiantes, instructores, empresas, cursos, proyectos. Al modelarlos como clases podemos reutilizar código, usar herencia y tener todo más ordenado conforme va creciendo la plataforma.
- **Evaluación y Seguimiento de progreso van mejor con funcional.** Aquí todo es transformar datos: entra una respuesta y sale una nota, entra el avance y sale un reporte. Al ser funciones puras, siempre dan el mismo resultado con la misma entrada, son fáciles de probar y no dan problemas, aunque tengamos miles de postulantes al año.
- **Colaboración en tiempo real va con eventos.** Las clases por Zoom, los chats y el trabajo en equipo tienen que reaccionar al instante cuando alguien hace algo, como mandar un mensaje, compartir pantalla o entrar a una sesión. Con eventos el sistema responde en ese momento, sin tener que seguir un flujo fijo ni recargar la página.

V. Buenas prácticas por capa de arquitectura:

5.1 Capa de presentación

1. **Separar la interfaz de la lógica:** la vista solo se encarga de mostrar cosas y de capturar lo que hace el usuario, nada de meterle reglas de negocio. Así si después hay que cambiar el diseño, no se toca todo el sistema.

2. **Usar el patrón Observer para el tiempo real:** cuando un estudiante manda un mensaje o un instructor comparte pantalla, los componentes que están observando ese evento se actualizan solos. Es lo que más nos sirve para la parte colaborativa de la plataforma.
3. **Reutilizar componentes:** hacer componentes que se puedan reutilizar como botones, tarjetas de curso o formularios. Así no repetimos código y toda la plataforma se ve igual, más consistente.

5.2 Capa de lógica de negocio

1. **Que cada cosa haga solo una cosa:** cada clase o función debería tener una sola razón para cambiar. Por ejemplo, una clase que solo calcule notas y otra aparte que arme los reportes, sin mezclar todo en una sola.
2. **Usar Factory para crear objetos:** cuando hay que crear varios tipos de usuario como estudiante, instructor o empresa, o tipos de contenido como curso o proyecto, es mejor tener una fábrica que se encargue de crearlos. Así si mañana se agrega un tipo nuevo, no hay que tocar todo el código**.**
3. **Que las funciones sean puras en evaluación y seguimiento:** en esa parte las funciones tienen que dar siempre el mismo resultado si les pasas los mismos datos, sin efectos raros por fuera. Eso hace que el código sea más predecible y mucho más fácil de probar.

5.3 Capa de datos

1. **Usar Singleton para la conexión a la BD:** en vez de abrir una conexión nueva a cada rato, se usa una sola instancia que maneja todo el acceso a los datos. Así no se duplican conexiones y rinde mejor cuando hay muchos usuarios conectados al mismo tiempo.
2. **Centralizar el acceso a los datos**: guardar todas las consultas en repositorios o servicios, para que la lógica del negocio no tenga que hablar directo con la base de datos. Si después se cambia de base, no se rompe todo lo demás.
3. **Validar y normalizar antes de guardar:** revisar que todo lo que entra, como correos, notas o perfiles, esté bien y en el formato correcto antes de guardarlo. Así se cuida que la información no se guarde mal y no dé problemas después.

- **<u>Conclusión:</u>**

Al final, analizando la plataforma de Laboratoria queda claro algo: no hay un paradigma o lenguaje que sea el mejor para todo. Lo que hace que un sistema sea bueno es saber cuándo usar cada cosa.

En este caso, orientado a objetos nos sirvió más para modelar a los actores de la plataforma, que son estudiantes, instructores y empresas, porque son entidades con estados y comportamientos y comparten una base en común. Lo funcional calzó mejor en evaluación y seguimiento, porque ahí lo que importa es transformar datos y que siempre dé el mismo resultado, sin efectos raros. Y lo de eventos era obvio que iba para la colaboración en tiempo real, porque el sistema tiene que reaccionar al instante a lo que hace cada usuario.

Con los lenguajes pasó lo mismo. Python destaca por todo lo de análisis de

datos y porque es fácil de aprender, Java por ser más estable y aguantar bien cuando hay mucha carga en empresas, y JavaScript porque es el único que corre directo en el navegador y te deja hacer full-stack con un solo lenguaje. Ninguno sobra, cada uno tiene su lugar según lo que necesite el módulo.

En conjunto, la idea no es ajustarse a un solo enfoque, sino mezclar lo mejor de cada paradigma y lenguaje para armar un sistema que pueda crecer sin volverse un desastre. Usando los pilares de orientado a objetos y patrones como Singleton, Factory y Observer, el código queda más mantenible, escalable y fácil de probar. Al final, es una plataforma que no solo funciona ahora, sino que ya está pensada para aguantar los más de 7.000 postulantes que tiene Laboratoria al año y todo lo que siga creciendo.

Referencias:

- Hub Laboratoria. (2024). La peruana que le apuesta a la formación de mujeres en tecnología.
- Programación Orientada a Objetos en JAVA. UPN.
- Xentic. (2024). Transformación Digital - Retos y Casos de Éxito en el Perú.